# ITA06 – MACHINE LEARNING

# ASSIGNMENT 2 – CO2

## NEURAL NETWORK REPRESENTATION, PERCEPTRONS, BACKPROPAGATION, DEEP LEARNING AND OPTIMIZATION

Student Name: A. Lakshmi Devi

Register Number: 192525176

Programme / Branch: AI & ML (Artificial Intelligence & Machine Learning)

Course Code: ITA0611

Course Name: Machine Learning

Assignment: 2 – CO2

Total Marks: 50

Duration: 1 Week

## CONTENTS

1. Q1 – Neural network representation and forward computation
2. Q2 – Perceptron learning and representational limitations
3. Q3 – MLP and backpropagation
4. Q4 – Shallow versus deep network comparison
5. Q5 – Overfitting and regularization
6. Q6 – Hyperparameter and optimization strategy
7. Q7 – Day-4 design challenge
8. Conclusion
9. Appendix – Python/NumPy implementation and experiment workflow

# INTRODUCTION

This assignment investigates neural-network design for manufacturing inspection and warehouse decisions. It covers binary classification, perceptron learning, multilayer perceptrons, backpropagation, depth, regularization and hyperparameter tuning. The answers follow the question order and evidence requirements in the supplied ITA06 Assignment 2 – CO2 brief.

Experimental honesty: the brief specifies experiments, plots and measured training times. These depend on running code in a particular Python environment. The report gives reproducible code and clearly marked analytical predictions, but does not invent observed metrics or claim that experiments have already run. Paste the actual console output and generated plots into the indicated places before submission.

# Q1 – DESIGN A NEURAL NETWORK REPRESENTATION FOR A CLASSIFICATION PROBLEM (6 MARKS)

## 1.1 Input features and numerical representation

| Feature | Meaning | Representation |
|---|---|---|
| x1 | Surface roughness | Measured real value in stated physical unit (e.g. µm). |
| x2 | Dimensional deviation | Signed real value from nominal dimension (e.g. mm). |
| x3 | Vibration level | Measured real value (e.g. mm/s or sensor unit). |
| y | Class label | Acceptable=0; Defective=1. |

Each input is converted to a numeric value. Fit a standard scaler on training data only: $z_j=(x_j-\mu_j)/\sigma_j$. Apply the same µ and σ to validation, test and production data. Scaling prevents a feature with a large numerical range from dominating the weighted sum and improves gradient-based optimization. Inspect missing values, sensor outliers and unit consistency before training.

## 1.2 Candidate architectures and parameter counts

Architecture A – single-layer logistic classifier: 3 input features connect directly to 1 sigmoid output. There is no hidden layer. The output estimates P(Defective=1|x). Trainable parameters = 3 weights + 1 bias = 4.

```
x1 ─┐\nx2 ─┼─> z = w1*x1 + w2*x2 + w3*x3 + b ─> sigmoid(z) ─> p(defective)\nx3 ─┘
```

Architecture B – one hidden layer: 3 input features → 4 hidden neurons (ReLU) → 1 sigmoid output. Input-to-hidden parameters = 3×4 weights + 4 biases = 16. Hidden-to-output parameters = 4×1 weights + 1 bias = 5. Total = 21 trainable parameters.

```
x1,x2,x3 ─> [ h1  h2  h3  h4 ] (ReLU) ─> [ output ] (sigmoid) ─> p
```

## 1.3 Activations and decision rule

Use ReLU, ReLU(u)=max(0,u), in the hidden layer because it introduces nonlinearity and is computationally simple. Use sigmoid $\sigma(z)=1/(1+e^{-z})$ in the output to produce a probability between 0 and 1. Classify as Defective when $p\geq 0.5$; otherwise Acceptable. The threshold can later be adjusted using validation data to reflect the cost of missed defects.

## 1.4 Worked forward pass for Architecture A

Use scaled illustrative input x=(0.5, −1.0, 1.5), weights w=(0.8, −0.4, 0.2), and bias b=−0.1.

$z = 0.8(0.5) + (-0.4)(-1.0) + 0.2(1.5) - 0.1 = 0.4 + 0.4 + 0.3 - 0.1 = 1.0.$

$p=\sigma(1)=1/(1+e^{-1})=0.7311$. Since $0.7311\geq 0.5$, predicted class = 1 (Defective). These values are a worked numerical example, not learned production parameters.

## 1.5 Architecture recommendation

If defect status depends on additive independent effects, Architecture A may be adequate and is easier to interpret. If defects occur only through interactions—for example, roughness is problematic only when vibration is also high—linear separation

may be insufficient. Architecture B can learn nonlinear interactions. Justify the hidden layer using a held-out validation comparison, stable improvement across multiple random seeds, confusion matrix/recall for defective parts, calibration and an inspection of errors. More parameters alone are not evidence that a hidden layer is necessary.

# Q2 – INVESTIGATE THE LEARNING BEHAVIOUR AND LIMITATIONS OF A PERCEPTRON (4 MARKS)

## 2.1 AND decision and truth table

| Packaging complete x1 | Quality passed x2 | Dispatch target y=x1 AND x2 |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

A single perceptron can learn AND because the positive point (1,1) can be separated by a straight line from the three negative points.

One valid final model is w1=1, w2=1, b=−1.5, with output 1 if net≥0. For (1,1), net=0.5 and output=1; for (1,0) or (0,1), net=−0.5 and output=0; for (0,0), net=−1.5 and output=0.

## 2.2 Perceptron learning configuration and first epoch

Use initial weights w=(0,0), bias b=0, learning rate η=0.2, threshold net≥0, and maximum 20 epochs. Online update for each sample: $\hat{y}=1$ if $w\cdot x+b\geq 0$ else 0; $e=y-\hat{y}$; $w\leftarrow w+\eta e\, x$; $b\leftarrow b+\eta e$.

| Input | Target | Prediction before training | Error | Weights/bias after this sample |
|---|---|---|---|---|
| (0,0) | 0 | 1 | −1 | (0,0), b=−0.2 |
| (0,1) | 0 | 0 | 0 | (0,0), b=−0.2 |
| (1,0) | 0 | 0 | 0 | (0,0), b=−0.2 |
| (1,1) | 1 | 0 | 1 | (0.2,0.2), b=0.0 |

This is the trace for the stated sample order and the convention net≥0. Subsequent epochs update until no errors or the epoch limit is reached. Because the AND function is linearly separable, the perceptron convergence theorem predicts convergence for a suitable learning setup and fixed finite dataset.

## 2.3 Changed requirement: exactly one indicator (XOR)

| x1 | x2 | XOR target |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

XOR is not linearly separable: the positive points (0,1) and (1,0) lie on opposite corners, while negative points (0,0) and (1,1) occupy the other two corners. No single straight decision boundary separates both classes.

Training the same perceptron at η=0.1 and η=0.5 may change update size and oscillation pattern, but neither learning rate can make a single linear threshold unit represent XOR. This is a model-representation limitation, not merely a training-configuration problem. A hidden layer with nonlinear activations is needed.

# Q3 – DEVELOP AND VERIFY BACKPROPAGATION FOR A MULTILAYER PERCEPTRON (6 MARKS)

## 3.1 Network and forward equations

Use 2 input neurons, 2 sigmoid hidden neurons and 1 sigmoid output neuron. For input $x=(x1,x2)$: $z_j=\Sigma_i w_{ij} x_i+b_j$; $h_j=\sigma(z_j)$; $z_o=\Sigma_j v_j h_j+c$; $\hat{y}=\sigma(z_o)$. Binary cross-entropy for target y is $L=-[y \ln(\hat{y})+(1-y)\ln(1-\hat{y})]$.

## 3.2 Gradients and updates

For sigmoid output with binary cross-entropy, output error $\delta_o=\partial L/\partial z_o=\hat{y}-y$. Therefore $\partial L/\partial v_j=\delta_o h_j$ and $\partial L/\partial c=\delta_o$. For hidden neuron j, $\delta_j=\delta_o v_j h_j(1-h_j)$. Thus $\partial L/\partial w_{ij}=\delta_j x_i$ and $\partial L/\partial b_j=\delta_j$. Gradient descent updates each parameter $\theta \leftarrow \theta-\eta\partial L/\partial\theta$.

The hidden activation must be nonlinear. Without it, a composition of linear layers is still one linear transformation and cannot represent XOR.

## 3.3 Manual one-example calculation

Illustrative initial parameters: $x=(1,0)$, $y=1$, input-to-hidden weights $W=[[0.1,-0.2],[0.2,0.1]]$ where columns are hidden neurons, hidden biases $b_h=(0,0)$, output weights $v=(0.3,-0.1)$, output bias $c=0$, learning rate $\eta=0.1$.

Hidden pre-activations: $z_{h1}=0.1$, $z_{h2}=-0.2$. Hidden outputs: $h1=\sigma(0.1)=0.5250$; $h2=\sigma(-0.2)=0.4502$.

Output pre-activation $z_o=0.3(0.5250)-0.1(0.4502)=0.1125$. Predicted output $\hat{y}=\sigma(0.1125)=0.5281$. Loss for $y=1$: $L=-\ln(0.5281)=0.6385$ (approximately).

Output delta $\delta_o=0.5281-1=-0.4719$. Gradients: $\partial L/\partial v1=-0.2477$; $\partial L/\partial v2=-0.2125$; $\partial L/\partial c=-0.4719$.

Hidden deltas: $\delta_{h1}=-0.4719(0.3)(0.5250)(1-0.5250)=-0.0353$; $\delta_{h2}=-0.4719(-0.1)(0.4502)(1-0.4502)=0.0117$. Since $x=(1,0)$, gradients for first input row are $(-0.0353,0.0117)$, and second input row gradients are zero. Bias gradients are $(-0.0353,0.0117)$.

Updated output weights: $v1=0.3248$; $v2=-0.0788$; $c=0.0472$. Updated first-row input weights: $W11=0.1035$; $W12=-0.2012$. Second-row weights remain unchanged for this example. Hidden biases become $(0.0035,-0.0012)$. Values are rounded; small differences may occur from rounding.

## 3.4 Numerical gradient validation

For a parameter $\theta$, central finite difference is $[L(\theta+\varepsilon)-L(\theta-\varepsilon)]/(2\varepsilon)$, with $\varepsilon\approx10^{-5}$. Compare this numerical estimate with the analytical gradient. A small absolute difference (for example below $10^{-4}$ on this small example) supports the implementation. Test multiple weights and biases.

Train on all four XOR patterns and plot loss against epoch. Because a small 2–2–1 sigmoid network can learn XOR, but initialization and learning rate affect convergence, report the actual final outputs and loss curve from the code. Do not claim success if the run does not converge.

# Q4 – INVESTIGATE WHETHER A DEEPER NETWORK IS A BETTER DESIGN (6 MARKS)

Use a reproducible 1,000-observation two-moons dataset with fixed random seeds. Split once into train/validation/test (e.g. 60%/20%/20%) and reuse exactly the same partitions for both architectures.

| Model | Architecture | Parameter count |
|---|---|---|
| Shallow | 2 → 16 (ReLU) → 1 (sigmoid) | (2×16+16)+(16×1+1)=65 |
| Deep | 2 → 16 → 16 → 16 → 1 | (2×16+16)+(16×16+16)+(16×16+16)+(16×1+1)=609 |

Before training, predict that both models can represent a curved boundary, but the three-hidden-layer model has greater representational capacity and may fit more complex patterns. It also has more parameters, greater computation and potentially more overfitting. The shallow network may be easier to train and may generalise equally well on a simple two-moons dataset.

Train each architecture with identical optimizer, learning rate, batch size, epoch budget, preprocessing and partitions. Run at least three seeds (e.g. 11, 22, 33). Record per seed: final train/validation loss, train/validation accuracy, test accuracy after selection, parameter count and training time. Plot learning curves and decision boundaries.

Architecture recommendation: select the shallow model if its validation performance is statistically comparable and its parameter count/training time are lower. Select deep only if it produces a consistent validation improvement across seeds and the test set confirms the selected model. An improvement cannot be attributed to depth alone when parameter counts differ (65 vs 609); capacity and depth are confounded. A parameter-matched comparison would better isolate depth.

## Q5 – DIAGNOSE OVERFITTING AND EVALUATE REGULARIZATION (7 MARKS)

Use the Q4 dataset and fixed validation/test partitions, but restrict the training subset to 100 labelled examples. Train a comparatively large network as the unregularized baseline. Expected behaviour: training loss may continue falling and training accuracy may approach 100%, while validation loss stops improving or rises and validation accuracy remains lower. This train-validation gap is evidence of overfitting.

| Configuration | Control | Expected effect / risk |
|---|---|---|
| Baseline | No regularization | Highest capacity to fit training data; likely larger generalization gap. |
| L2 | $\lambda$=0.001 added to loss as $\lambda/2\ \Sigma w^2$ | Penalises large weights; smoother decision boundary. Too large $\lambda$ can underfit. |
| Dropout | Dropout rate 0.25 on hidden activations during training | Reduces co-adaptation; may improve validation performance. Too much dropout can underfit. |
| Early stopping | Monitor validation loss; patience=20; restore best weights | Stops after validation loss fails to improve; avoids late overfit. |

Keep the architecture, data split and other training settings fixed when comparing each intervention. Select the best configuration using validation loss and validation recall/accuracy, not training accuracy. Evaluate the chosen configuration on the test set only once. Higher training accuracy does not necessarily mean a better model: generalisation to unseen components is the goal.

The most useful intervention must be decided from the actual validation curves. L2, dropout and early stopping act differently, so do not assert a winner without running the controlled experiment. Underfitting is indicated when both training and validation performance are poor.

## Q6 – ENGINEER A HYPERPARAMETER AND OPTIMIZATION STRATEGY (8 MARKS)

The budget permits exactly 12 candidate runs. Use the same Q4 dataset, fixed partitions, preprocessing, maximum epochs and stopping rule. Use validation loss as the primary selection measure, with validation accuracy as a secondary measure. Keep the test set untouched until final selection.

| Run | Optimizer | Learning rate | Batch size | Hidden width | Purpose |
|---|---|---|---|---|---|
| 1 | SGD | 0.01 | 32 | 16 | Baseline |
| 2 | SGD | 0.001 | 32 | 16 | Isolate lower LR |
| 3 | SGD | 0.1 | 32 | 16 | Test high LR |
| 4 | SGD + momentum 0.9 | 0.01 | 32 | 16 | Optimizer comparison |
| 5 | Adam | 0.001 | 32 | 16 | Optimizer comparison |
| 6 | Adam | 0.01 | 32 | 16 | Adam LR comparison |
| 7 | Adam | 0.001 | 16 | 16 | Batch-size comparison |
| 8 | Adam | 0.001 | 64 | 16 | Batch-size comparison |
| 9 | Adam | 0.001 | 32 | 8 | Width comparison |
| 10 | Adam | 0.001 | 32 | 32 | Width comparison |
| 11 | SGD + momentum 0.9 | 0.01 | 16 | 16 | Combined candidate |
| 12 | Adam | 0.001 | 16 | 32 | Combined candidate |

Runs 1–3 isolate learning rate under SGD. Runs 1, 4 and 5 compare optimization strategies at comparable settings. Runs 5, 7 and 8 investigate batch size; runs 5, 9 and 10 investigate width. Runs 11–12 are combined candidates and should be interpreted as such, not as isolated one-variable comparisons.

A learning rate that is too small causes slow convergence and may not reach a useful solution within the epoch budget. A learning rate that is too large can overshoot minima, produce oscillating loss or diverge. SGD updates using the current mini-batch gradient; momentum accumulates a moving direction and can accelerate progress; Adam adapts per-parameter step sizes using first and second moment estimates.

Record configuration, best validation score, epoch of best score, training time and any instability. Choose the lowest validation loss (or best agreed validation metric) subject to acceptable training time. Report test performance only after choosing the configuration. The 12-run budget is limited, so it cannot exhaustively explore all interactions. Test data must remain outside tuning because repeated test-based decisions leak information and bias the final estimate.

## Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

### A. Diagnose the learning or computational bottleneck (2 marks)

Selected limitation: the single perceptron fails on the XOR review flag in Q2. This is primarily a model-representation bottleneck, not a learning-rate problem. Evidence: XOR's positive points (0,1) and (1,0) cannot be separated from negative points (0,0) and (1,1) by one line. Training at different learning rates changes the path of updates but cannot create a nonlinear decision boundary.

### B. Define the design objective (2 marks)

Baseline: single perceptron on XOR, which cannot represent the target and therefore cannot achieve zero classification error on all four combinations. Measurable objective: implement a 2–2–1 MLP with sigmoid hidden and output units and achieve correct predictions on all four XOR patterns (100% training accuracy) with decreasing binary cross-entropy. Also verify analytical gradients against finite differences.

### C. Design the revised learning strategy (4 marks)

Replace the single linear threshold unit with a 2-input, 2-hidden-neuron, 1-output MLP. Use sigmoid activations in hidden and output layers, binary cross-entropy loss, backpropagation and mini-batch or full-batch gradient descent. The hidden layer creates intermediate nonlinear features that make XOR separable in the transformed representation.

```
Input x1, x2
      |
      ▼
Hidden layer: 2 sigmoid neurons
      |
      ▼
Output layer: 1 sigmoid neuron
      |
      ▼
p(XOR=1), classify at threshold 0.5

For each epoch:
  forward: h = sigmoid(X @ W1 + b1)
           p = sigmoid(h @ W2 + b2)
  loss: binary cross entropy(y, p)
  backward: compute output delta and hidden delta
  update W1, b1, W2, b2 using learning rate η
  record loss and predictions
```

The parameter count is (2×2+2)+(2×1+1)=9. This is a small computational cost compared with a deep model. The trade-offs are sensitivity to initialization and learning rate, and less direct interpretability than a single threshold rule.

### D. Predict before implementing (1 mark)

Prediction: the MLP will represent XOR and reach zero training classification error on the four patterns for a suitable initialization and training configuration. The key expected improvement is representational ability and classification accuracy, with loss decreasing during training.

### E. Implement and validate the redesign (2 marks)

Use identical four XOR examples for the perceptron and MLP. Compare predictions, accuracy, loss and parameter count. Check gradients numerically. Use the same random seed and record training settings. The code appendix provides a NumPy implementation and prints final predictions; run it to capture the actual result.

### F. Justify the final design (2 marks)

The redesign is justified if the MLP produces the XOR truth table correctly and its loss decreases while gradient checks pass. The perceptron remains preferable for linearly separable AND dispatch because it is simpler, faster and easier to explain. Use the MLP only for the nonlinear XOR review flag. If training fails, inspect gradient checks, initialization and learning rate before concluding that the MLP representation is inadequate.

## CONCLUSION

A single logistic unit is a compact model for linearly separable manufacturing decisions, and a perceptron can learn AND. XOR demonstrates the representational limitation of a single linear threshold unit. A 2–2–1 MLP introduces nonlinear features and backpropagation provides a systematic way to update its parameters. Deeper networks offer greater capacity but must be justified by validation evidence. Regularization and disciplined hyperparameter tuning help control overfitting and computational cost. The final design should be selected using validation data, with the test set reserved for a single final evaluation.

## APPENDIX – PYTHON / NUMPY IMPLEMENTATION

```
# Python 3.x, NumPy only. Manual forward/backprop for XOR.
import numpy as np
np.set_printoptions(precision=4, suppress=True)

X = np.array([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y = np.array([[0.],[1.],[1.],[0.]])
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -40, 40)))
def bce(y, p):
    p = np.clip(p, 1e-9, 1-1e-9)
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

rng = np.random.default_rng(7)
W1 = rng.normal(0, 0.5, (2,2)); b1 = np.zeros((1,2))
W2 = rng.normal(0, 0.5, (2,1)); b2 = np.zeros((1,1))
lr = 0.8
losses = []
for epoch in range(20000):
    # Forward
    H = sigmoid(X @ W1 + b1)
    P = sigmoid(H @ W2 + b2)
    loss = bce(y, P); losses.append(loss)
    # Backprop for mean BCE + sigmoid output
    dZ2 = (P-y) / len(X)
    dW2 = H.T @ dZ2; db2 = np.sum(dZ2, axis=0, keepdims=True)
    dH = dZ2 @ W2.T
    dZ1 = dH * H * (1-H)
    dW1 = X.T @ dZ1; db1 = np.sum(dZ1, axis=0, keepdims=True)
    W2 -= lr*dW2; b2 -= lr*db2
    W1 -= lr*dW1; b1 -= lr*db1

H = sigmoid(X @ W1 + b1)
P = sigmoid(H @ W2 + b2)
print("Final loss:", bce(y,P))
print("Probabilities:", P.ravel())
print("Predictions:", (P >= 0.5).astype(int).ravel())
print("Targets:", y.ravel())

# Numerical gradient check for W2[0,0]
def loss_for(W2_test):
    hh = sigmoid(X @ W1 + b1)
    pp = sigmoid(hh @ W2_test + b2)
```

```
    return bce(y, pp)
eps = 1e-5
Wp = W2.copy(); Wm = W2.copy()
Wp[0,0] += eps; Wm[0,0] -= eps
numeric = (loss_for(Wp)-loss_for(Wm))/(2*eps)
hh = sigmoid(X @ W1 + b1); pp = sigmoid(hh @ W2 + b2)
analytic = np.sum(hh[:,0:1] * (pp-y) / len(X))
print("Gradient check W2[0,0]:", numeric, analytic)
```

For Q4–Q6, use the same dataset split and a fixed random seed for every architecture/configuration. Record the actual plots and metric tables from execution. The assignment brief does not supply precomputed outcomes, so this report intentionally distinguishes analytical predictions from results that must be measured.